



POLITECNICO
MILANO 1863

COMPUTER SCIENCE AND ENGINEERING
SOFTWARE ENGINEERING II
2025-2026

Done By:

Nafis Ismam - 11117431

Ayman Mansur - 11119831

Christopher Khoury - 11085985

Design Document (DD)

Contents

1. Introduction

- 1.1 Purpose
- 1.2 Scope
- 1.3 Definitions, Acronyms, Abbreviations
- 1.4 Revision History
- 1.5 Document Structure

2. Architectural Design

- 2.1 Overview: High-level components and their interaction
- 2.2 Component view
- 2.3 Deployment view
- 2.4 Runtime view
- 2.5 Component interfaces
- 2.6 Selected architectural styles and patterns
- 2.7 Other design decisions

3. User Interface Design

4. Requirements Traceability

5. Implementation, Integration & Testing

- 5.1 Overview
- 5.2 Implementation Plan
- 5.3 Component Integration and Testing
- 5.4 System Testing

6. Effort Spent

7. References

1. Introduction

1.1 Purpose

The primary objective of this document is to discuss the comprehensive architectural, functional, and data-centric specification of the Best Bike Paths(BBP) software system. This document serves as the authoritative technical blueprint for the development, testing, and eventual deployment of a robust digital ecosystem designed to support the urban cycling community through a high-fidelity, crowdsourced data system.

The Best Bike Paths project was born out of a persistent technological gap in infrastructure management and a significant global shift in urban mobility paradigms. The bicycle has become a key component of contemporary urban planning as cities increasingly adopt Active Transportation (AT) models to reduce traffic, lower carbon emissions, and improve public health. However, a "safety perception gap" and a lack of accurate, detailed information about the state, rather than just the existence of cycling infrastructure, often disrupt the successful adoption of cycling as a primary mode of transportation. While navigation services such as Google Maps frequently fail to capture the dynamic attributes that determine a route's safety and comfort of the cyclist, such as surface roughness, temporary obstacles (e.g ,potholes, constructions).

The purpose of Best Bike Paths (BBP) is to address this particular information asymmetry. The goal of this "Biker's Association" project is to enable the cycling community as a whole to keep an active inventory of the road system. Beyond just route planning, the system serves as a platform for Participatory Sensing, in which each cyclist serves as a mobile sensor node and adds to a shared knowledge base that makes the community's invisible road hazards visible.

The core purpose of the BBP system can be decomposed into three strategic pillars:

1. Safety Centric Navigation via Quality Metrics: Unlike traditional routing engines that optimize primarily for distance or time, BBP prioritizes the quality of the ride. By implementing user-generated data on path status (ranging from "Optimal" to "Requires Maintenance") and specific hazards, the system enables risk-averse routing. This is particularly crucial for bridging the gap between "confident cyclists" who are comfortable in traffic and the wider population that requires "All Ages and Abilities" (AAA) infrastructure to feel safe. The system provides the mechanism to visualize and navigate based on a computed "Path Score," effectively filtering out technically legal but practically dangerous routes.
2. The Democratization of Infrastructure Monitoring: Municipalities often lack the resources to continuously monitor the micro-condition of every bike lane. Professional surveying vehicles equipped with LiDAR and high-resolution cameras are effective but

prohibitively expensive for daily monitoring. BBP democratizes this process by utilizing "Automated Mode" data acquisition. By leveraging the sensors already present in users' commercial off-the-shelf (COTS) smartphones, specifically the accelerometer, gyroscope, and GPS, the system creates a cost-effective, continuous monitoring network. This "Smart Sensing" capability allows for the automatic detection of surface anomalies (potholes) during regular commutes, dramatically increasing the temporal resolution of infrastructure data compared to annual municipal audits.

In summary, the purpose of the BBP system is to transform the solitary act of cycling into a collaborative data-generation event, creating a Digital Twin of the cycling infrastructure that is richer, more dynamic, and more safety-oriented than any existing static map. This document details the software architecture required to support these high-level goals, addressing the specific challenges of mobile telemetry, sensor fusion, temporal data merging, and geospatial visualization.

1.2 Scope

The Best Bike Paths (BBP) system's scope outlines the parameters of the software solution, including what the system will do, the data it will process, and the limitations it must adhere to. The project includes the complete development of a distributed client-server application, which consists of a robust backend for data processing, storage, and reconciliation, and a mobile frontend for data acquisition and user interaction.

Each of the four main subsystems that make up the scope functionally addresses a different aspect of the "Best Bike Paths" problem statement. (1) The Crowdsourced Inventory & Smart Sensing Module; (2) The Quantified Self & Telemetry Module; (3) The Merging & Truth Maintenance Engine; and (4) The Context-Aware Visualization & Routing Module.

1.3 Definitions, Acronyms, Abbreviations

This section provides a comprehensive glossary of the technical, domain-specific, and project-specific terminology used throughout this Design Document. A precise shared vocabulary is essential for aligning the software engineering team with the domain requirements of urban mobility and GIS (Geographic Information Systems).

Acronym	Meaning	Description
AAA	All Ages and Abilities	A cycling infrastructure planning standard referring to routes safe for users of all ages and skill levels. In BBP, the <i>Optimal</i> tag approximately corresponds to AAA-compliant routes.
API	Application Programming Interface	A set of protocols enabling communication between the BBP mobile client, backend server, and external services such as weather providers and map tile servers.
BBP	Best Bike Paths	The name of the software system described in this Design Document.
COTS	Commercial Off-The-Shelf	Refers to standard consumer smartphones (Android/iOS) used as sensing devices instead of specialized surveying hardware.
CRUD	Create, Read, Update, Delete	The four fundamental operations supported by persistent storage systems, applied to user profiles, trip logs, and path metadata in BBP.
DD	Design Document	The document describes the system architecture, components, interfaces, and detailed design of BBP.
GDPR	General Data Protection Regulation	The European Union regulation governing personal data protection. It is particularly relevant to BBP due to the handling of sensitive geospatial and mobility data.
GIS	Geographic Information System	A framework and set of tools used to capture, store, analyze, and manage spatial and geographic data within the system.
GPS	Global Positioning System	A satellite-based navigation system used by the mobile client to track user position, speed, and movement trajectory.

GUI	Graphical User Interface	The visual interface through which users interact with BBP on mobile and web platforms.
HITL	Human-in-the-Loop	A design approach in which human validation is required for certain automated decisions. In BBP, it is used to confirm anomalies such as suspected potholes.
IMU	Inertial Measurement Unit	A combination of accelerometers and gyroscopes embedded in smartphones, used for detecting road surface irregularities.
JSON	JavaScript Object Notation	A lightweight data-interchange format used for REST API requests and responses, such as path data and weather information.
MADT	Monthly Average Daily Traffic	A traffic volume metric commonly used in transport planning. While BBP relies on crowdsourced data, this concept aids in interpreting traffic-related indicators.
OD	Origin-Destination	A transportation planning term indicating the starting point and endpoint of a trip.
RASD	Requirement Analysis and Specification Document	The document preceding the Design Document, defining the functional and non-functional requirements of BBP.
REST	Representational State Transfer	An architectural style for designing networked applications, adopted by the BBP backend services.
SDK	Software Development Kit	A collection of development tools and libraries (e.g., mapping and location services SDKs) integrated into the BBP mobile application.
UML	Unified Modeling Language	A standardized modeling language used to describe the structure and behavior of the system through diagrams such as class, sequence, and component diagrams.

Definitions:

Accelerometer Data: Time-series data retrieved from the mobile device measuring proper acceleration (m/s^2). In the context of BBP, the system monitors the Z-axis (vertical) acceleration. High-frequency, high-amplitude spikes in this data stream are processed to identify potential surface irregularities. The design must account for the device's orientation relative to gravity.

Automated Mode: A data acquisition state where the BBP application operates as a background service. It utilizes heuristic algorithms to detect the onset of cycling activity (typically via speed thresholding, e.g., 10 km) and automatically logs the path and inertial anomalies. Crucially, this mode requires post-hoc user verification to filter false positives.

Bayesian Truth Serum: A theoretical framework often used in crowdsourcing to score the truthfulness of an answer based on how surprisingly common it is. While BBP may use a simpler "Majority Vote" consensus, the concept of "Truth Maintenance" relies on weighting user inputs to predict the actual state of the world.

Bike Path (System Definition): Within BBP, a "Bike Path" is defined functionally rather than strictly legally. It encompasses:

1. Physically separated cycle tracks or lanes.
2. Shared roadways where vehicular traffic is rare, and speed limits are compatible with cycling average speeds.

This definition allows the system to map "cycling-friendly" networks that extend beyond official infrastructure, relying on user perception of safety.

Consensus Algorithm: The logic implemented in the Merging Engine to resolve conflicting data reports. It determines the "Display State" of a path segment by aggregating multiple user inputs. The algorithm prioritizes agreement (multiple users reporting the same status) and penalizes outliers, while also factoring in the decay of information value over time.

Crowdsourcing: A data collection paradigm where the general public contributes data to a central repository. BBP utilizes both "Participatory Sensing" (users actively and manually tagging paths) and "Opportunistic Sensing" (automated background collection via sensors where the user's primary intent is just to ride).

Freshness (Temporal Decay): A critical attribute of data validity in the BBP system. "Freshness" refers to the age of a data point. The system employs a Time Decay Function where the weight of a report decreases as time elapses. This ensures that the system's map reflects the current state of the road (e.g., acknowledging that a pothole reported last year may have been repaired).

Geofencing: A virtual geographic boundary. BBP may use geofencing to trigger "Automated Mode" confirmation prompts when a user arrives at their destination or enters a known "poor quality" zone.

Map Matching: The algorithmic process of aligning a sequence of noisy GPS coordinates (the recorded trace) to the nearest logical road segment in the digital map database. Raw GPS data often drifts due to

signal reflection (multipath errors). Map Matching ensures that attributes like "Pothole" or "Smooth Surface" are associated with the permanent Road ID rather than an ephemeral coordinate.

Path Score: A composite numerical value calculated for each path segment. It serves as the primary metric for the routing engine. The score is derived from:

1. **Status Attributes:** (Optimal, Medium, Sufficient, Requires Maintenance).
2. **Impedance Factors:** Presence of obstacles (potholes).
3. **Efficiency:** The topological directness of the route.

Visualizations use this score to color-code maps (e.g., Green/Yellow/Red)

Pothole (Anomaly): In the BBP system, a "Pothole" is a specific classification of obstacle detected via sharp vertical acceleration forces. It represents a degradation of road surface quality that poses a safety risk to cyclists.⁵

Snapping: A GIS operation where a point (e.g., a user-dropped pin for a pothole) is moved to exactly coincide with a line (the road centerlines). This ensures topological consistency in the database.

1.4 Revision History

This table tracks the document’s history and the changes made over time.

Version	Date	Author(s)	Summary
1.0	21/12/2025	Nafis Ismam	Introduction and Initial Draft.
1.1	22/12/2025	Ayman Mansur	First draft of Architectural Design
1.2	24/12/2025	Ayman Mansur	UI design draft
1.3	25/12/2025	Nafis Ismam	Requirement tracing draft
1.4	25/12/2025	Ayman Mansur	UI design description draft
1.5	27/12/2025	Nafis Ismam	Implementation, integration and test plan integration
1.6	02/01/2026	Nafis Ismam	Requirement tracing final draft
1.7	03/01/2026	Ayman Mansur	Diagrams refined, sequence diagrams updated, UI design finalised and edited descriptions
2.0	05/01/2025	Nafis Ismam	Merged contributions and edited for consistency.

1.5 Reference Documents

[1] N. Ismam, A. Mansur, C. Khoury, "Requirement Analysis and Specification Document (RASD) - Best Bike Paths," Version 2.0, Dec 2025.

1.6 Document Structure

This Design Document (DD) is organized into seven main sections, following the structure provided for the course project.

1. Introduction: This section outlines the document's objectives, purpose, and scope. It serves as a bridge from the Requirement Analysis and Specification Document (RASD), reiterating the primary goals and architectural drivers of the Best Bike Paths (BBP) system. It also contains the definitions, acronyms, revision history, and a listing of reference documents used to frame the design.

2. Architectural Design: This section presents a comprehensive view of the system's architecture, moving from high-level abstractions to detailed component specifications. It describes the system's decomposition into subsystems and their interactions. It is further categorized into:

High-Level Components: An overview of the main subsystems and their relationships.

Component View: A detailed description of individual components and their interfaces.

Deployment View: How the software artifacts are mapped onto hardware nodes.

Runtime View: Diagrams and descriptions illustrating how the system behaves during execution.

Selected Architectural Patterns: The rationale behind the specific design patterns chosen to satisfy the system's requirements.

3. User Interface Design: This section details the frontend experience and visualizes the interaction between the user and the BBP system. It presents the user interface design through mockups, wireframes, and navigational flow diagrams. It demonstrates how the functional requirements are translated into an intuitive graphical user interface (GUI) for both the mobile application and the web platform.

4. Requirements Traceability: This section ensures alignment between the analysis and design phases. It provides a detailed mapping that links the architectural components and design decisions back to the specific functional and non-functional requirements defined in the RASD. This establishes that every design element addresses a specific system goal and that all requirements are adequately covered by the proposed architecture.

5. Implementation, Integration, and Test Plan: This section outlines the strategy for realizing the software system. It describes the planned sequence of component implementation and the methodology for integrating these components into a cohesive system. Furthermore, it details the testing procedures—ranging from unit testing to system integration validation—required to ensure the platform meets its quality and reliability standards.

6. Effort Spent: This section contains a report detailing the number of hours each group member has worked to complete this document. It breaks down the contributions regarding architectural modeling, diagram creation, and documentation.

7. References: This final section lists any external documents, standards, software libraries, or tools cited within this specification.

2. Architectural Design

2.1 Overview

This section provides a high-level description of the architecture of the **Best Bike Paths (BBP)** system, including its main components and how they interact across architectural layers. The architectural choices aim to satisfy the functional and non-functional requirements defined in the RASD, with particular focus on: (i) mobile trip recording with GPS data acquisition, (ii) manual and automated insertion of bike path information, (iii) enrichment of recorded trips with meteorological data, and (iv) searching and visualizing published bike paths on a map.

BBP is primarily accessed through a **mobile application**, which is the preferred platform for trip recording since GPS and related motion data are directly collected from smartphone sensors. The mobile client communicates with the server-side application tier via **HTTPS** and **REST APIs**. External services (map provider, weather provider, push notifications) are accessed through an intermediary proxy, in order to centralize security checks, logging, and rate-limit handling.

2.1.1 System View

The BBP system is designed according to a **4-tier client-server architecture**:

1. **Presentation Layer (Client Tier)** : the mobile application used by cyclists and other users.
2. **Application Layer - Front-End Tier** : entry-point components handling routing, authentication checks, and request management.
3. **Application Layer - Back-End Tier** : core services implementing system functionalities (organized as microservices).
4. **Data Layer (Data Tier)** : persistent storage and data management.

The information flow starts from the mobile application and reaches the back-end services through the API Gateway. Requests that require external providers (e.g., map tiles/overlays, weather data, push notifications) are routed through a dedicated proxy. Responses are then formatted (typically JSON payloads) and returned to the mobile application for rendering.

A **microservices architecture** is adopted in the back-end tier to keep responsibilities separated and allow independent scalability. For instance, trip ingestion can be scaled independently from path search and visualization. The API Gateway enables horizontal scaling by dispatching requests to available service replicas and provides a centralized control point for authentication, rate limiting, and request validation. **Figure 1** shows the overall architecture of BBP and the interaction among tiers.

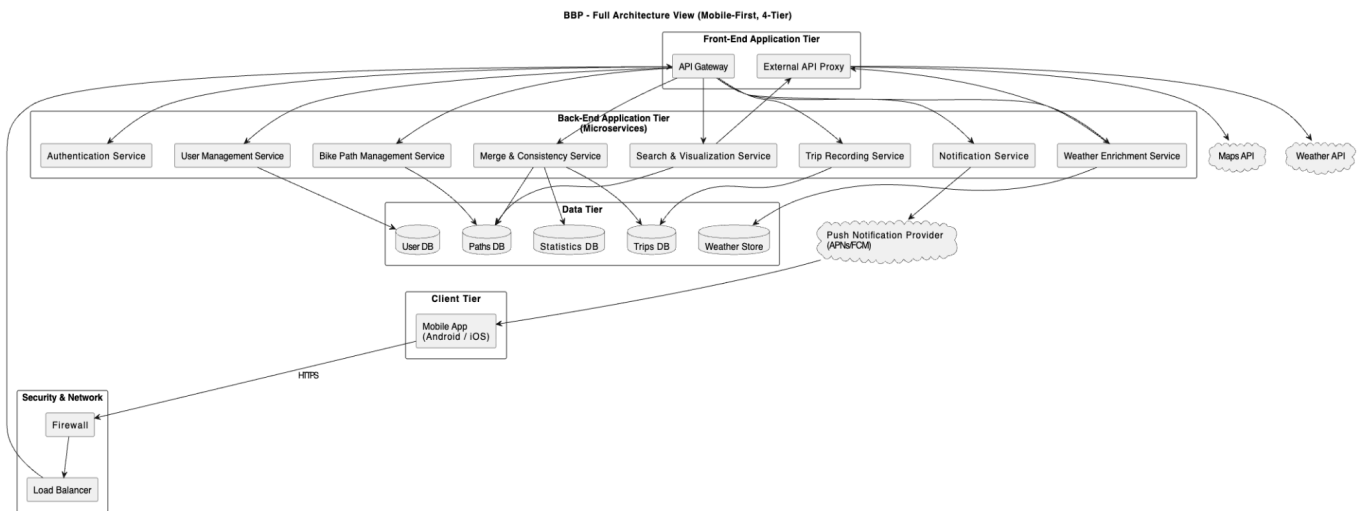


Figure 1 – BBP Full Architecture View (Mobile-First, 4-Tier).

2.1.2 Detailed View

Presentation Tier (Mobile Client)

The user interacts with BBP through an Android/iOS mobile application. The app is responsible for:

- user authentication and profile access;
- trip recording (collecting GPS points over time);
- sending trip segments/candidate paths for automated insertion (when enabled);
- manual insertion of bike path information (streets/segments, status, obstacles, publishability);
- searching and visualizing bike paths on a map;
- showing trip statistics and weather enrichment results.

The mobile application communicates with the system through HTTPS requests and receives responses containing the required data for visualization.

Application Tier – Front-End

This tier contains components that sit at the boundary between client requests and core services:

- **API Gateway**, responsible for routing requests to the appropriate microservice, performing authentication/authorization checks, rate limiting, and optionally caching frequent requests;
- **External API Proxy**, responsible for managing calls to third-party services (maps provider, weather provider, push provider). This component centralizes logging, response transformation, and protection against API rate limits.

This tier increases security by reducing the exposure of internal services and by filtering/validating requests before they reach the core services.

Application Tier – Back-End (Microservices)

The back-end application tier hosts the core business logic of BBP. Services are implemented as microservices that expose RESTful endpoints and interact with each other as needed. The back-end is responsible for:

- processing and storing recorded trips and associated statistics;
- managing manual and automated bike path insertion workflows;
- retrieving meteorological data and associating it with recorded trips;
- providing search and visualization features, including path filtering;
- merging and consolidating information from multiple sources (manual reports and recorded traces);
- triggering push notifications when needed (e.g., confirmation requests, updates).

Data Tier

The data tier contains the persistent storage used by BBP. The databases are deployed in a replicated configuration to ensure availability and to reduce the risk of data loss. At a logical level, the system stores:

- user data and preferences;
- recorded trips and GPS points;
- published and non-published path data (status, obstacles, metadata);
- weather data or cached weather responses;
- aggregated statistics used for visualization and search optimization.

2.2 Component View

This section describes the main internal components of BBP and their responsibilities. Components are grouped into client-side, server-side, and external services.

Back-End Components

- **Authentication Service**
Manages login and token validation for registered users. It issues or validates access tokens and supports secure access to protected endpoints.
- **User Management Service**
Manages user profiles and preferences (e.g., visibility settings, publishability defaults). It provides methods for updating user data and retrieving profile information.
- **Trip Recording Service**
Receives GPS points collected by the mobile application during a trip. It stores trip points, computes statistics (distance, duration, average speed), and persists the trip summary. It can also support automated mode by detecting biking-like segments based on telemetry.
- **Bike Path Management Service**
Handles bike path information insertion. It supports:
 - **manual insertion**, where the user specifies streets/segments, status, known obstacles, and publishability;
 - **automated insertion**, where candidate segments derived from recorded telemetry are confirmed by the user before becoming publishable path information.

- **Weather Enrichment Service**
Integrates meteorological information into recorded trips. It retrieves weather data from an external provider (through the proxy), and associates it with trips when available.
- **Search & Visualization Service**
Implements path search and retrieval based on user filters (e.g., area, status, obstacles). It provides map-ready representations of results and coordinates with the maps provider via the proxy.
- **Merge & Consistency Service**
Consolidates and merges path information coming from different sources (e.g., multiple recordings of the same segment and manual reports). It also maintains aggregated indexes and statistics used for search and visualization.
- **Notification Service**
Generates and sends notifications, primarily through a push notification provider (APNs/FCM). It is used for events such as confirmation prompts in automated mode or important system updates.
- **DBMS / Data Access Layer**
Encapsulates database access for all services, providing a controlled and consistent interaction with persistent storage.

Client-Side Components

- **BBP Mobile App**
The main user-facing component. It provides the interface for trip recording, insertion workflows, and map visualization. It also interfaces with smartphone GPS sensors to collect telemetry.

External Services

- **Maps API**
Used for base maps, overlays, and map-related services required by path visualization.
- **Weather API**
Provides meteorological data needed for trip enrichment.
- **Push Notification Provider (APNs/FCM)**
Used to deliver notifications to user devices.

Figure 2 shows the component diagram of BBP and the main dependencies among components.

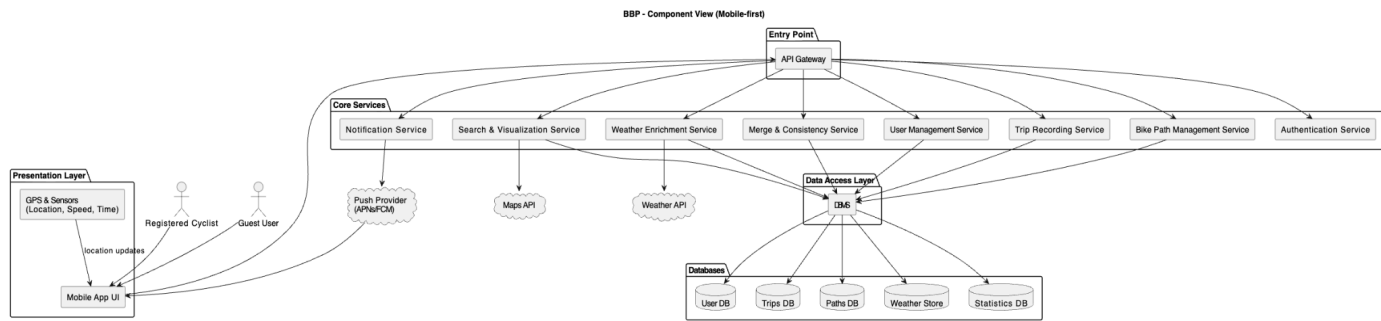


Figure 2 – component diagram of BBP

2.3 Deployment View

This section describes the execution environment of BBP and how software is distributed across hardware nodes. Users access BBP through an Android/iOS mobile application. The smartphone collects GPS data locally and sends it to the server using HTTPS. The server-side environment is deployed on a cloud infrastructure composed of replicated entry-point components (API Gateway) and replicated microservices. The data layer is hosted on a replicated database cluster to guarantee availability and fault tolerance.

A typical deployment includes:

- **Mobile Device** running the BBP mobile app and GPS sensors.
- **Edge/DMZ** including firewall and load balancer.
- **Application Cluster** hosting the API gateway and microservices.
- **Database Cluster** hosting replicated databases.
- **External providers** for maps, weather data, and push notifications.

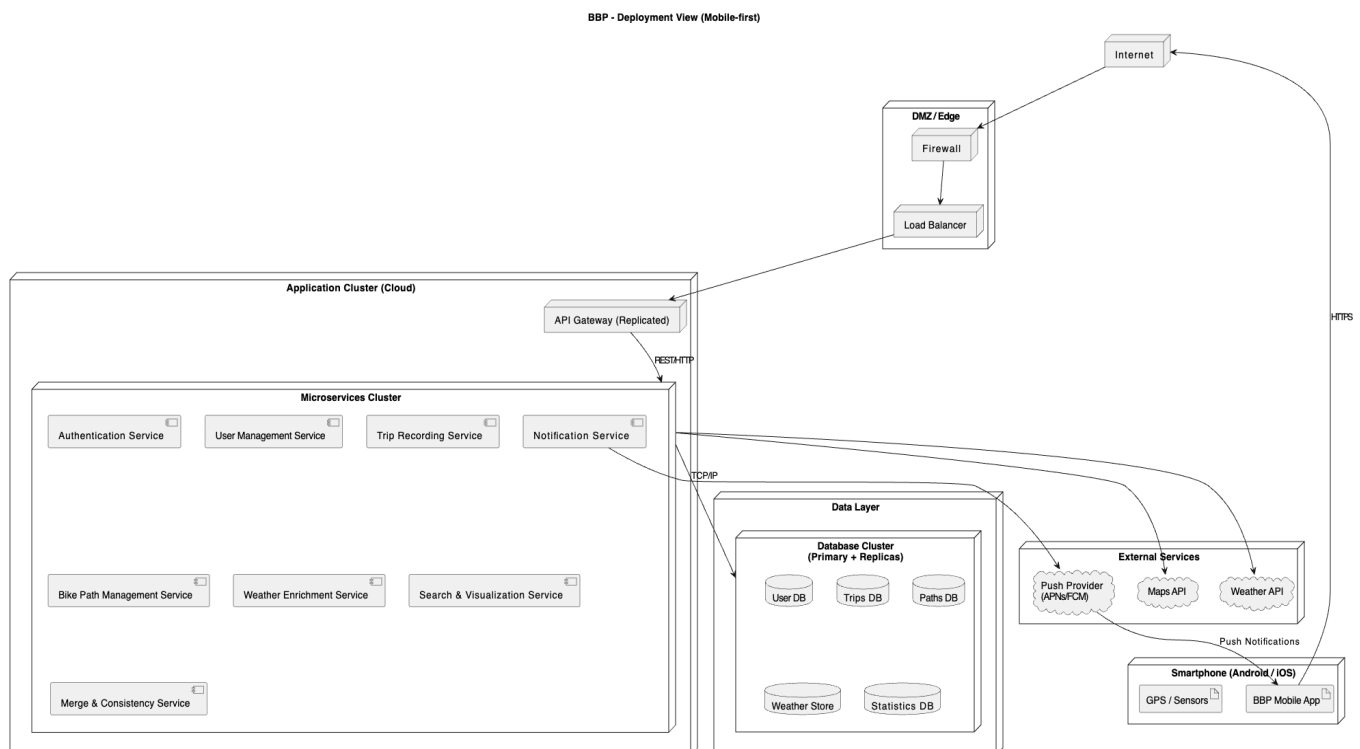


Figure 3 – Deployment view of BBP

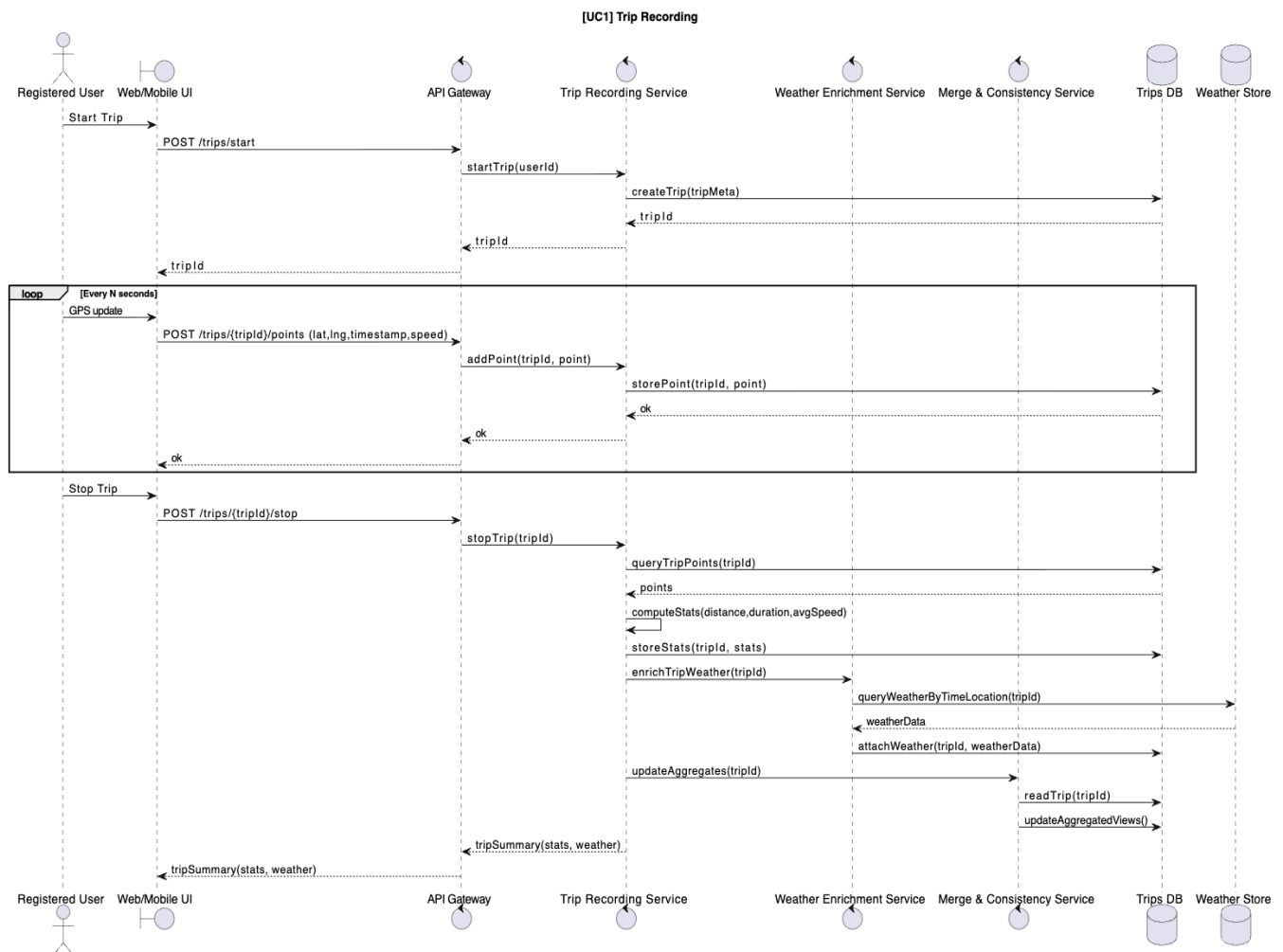
2.4 Runtime View

This section describes the flow of messages among components using sequence diagrams. For simplicity, authorization checks, caching mechanisms, and error handling (e.g., external API failures) are omitted. The sequence diagrams focus on the main logical flow needed to complete each use case.

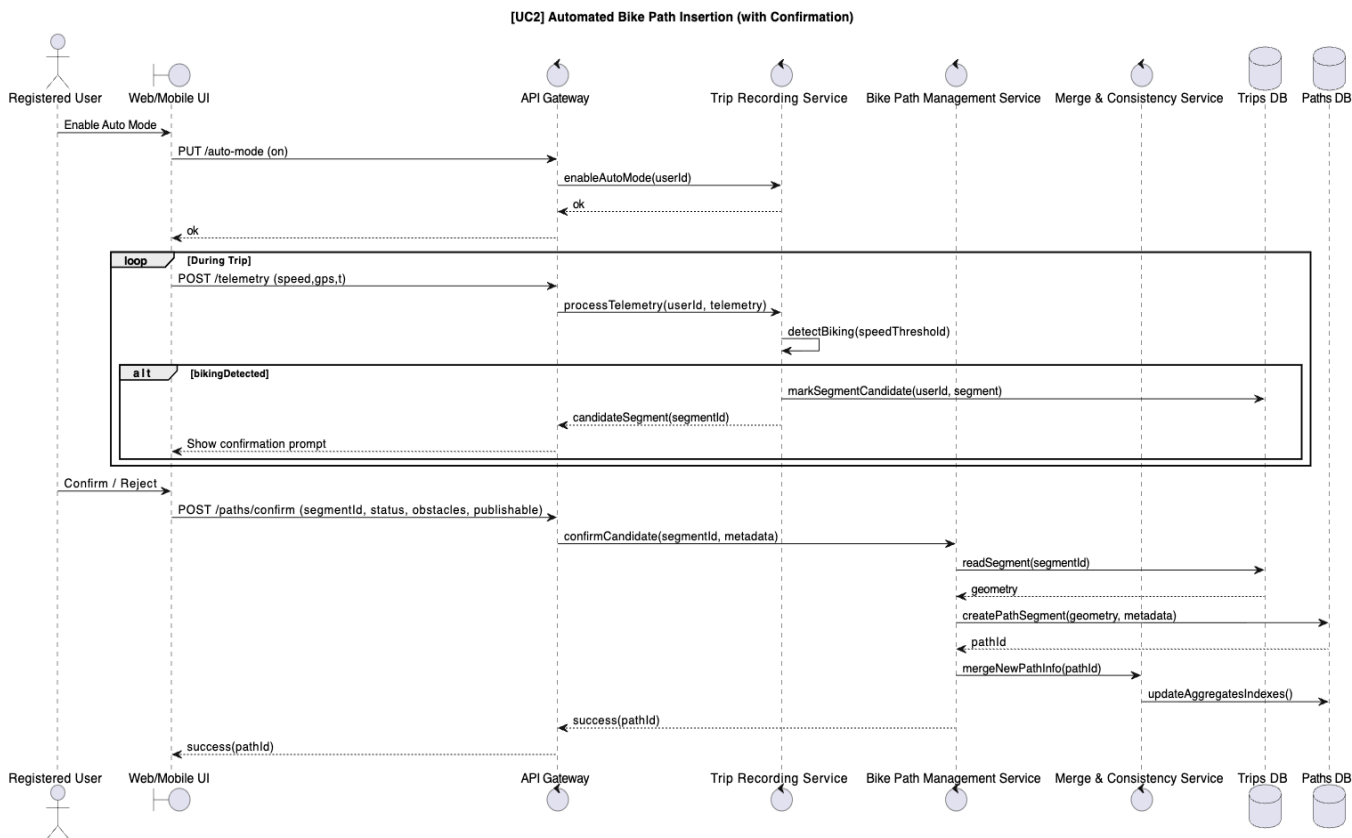
The following runtime scenarios are considered:

- **Trip Recording** (GPS collection, storage, statistics computation, weather enrichment);
- **Automated Insertion with Confirmation** (telemetry-based detection and user confirmation);
- **Manual Insertion** (user-provided path data storage);
- **Path Search & Visualization** (filters + map rendering);
- **Information Merging** (periodic consolidation and conflict resolution).

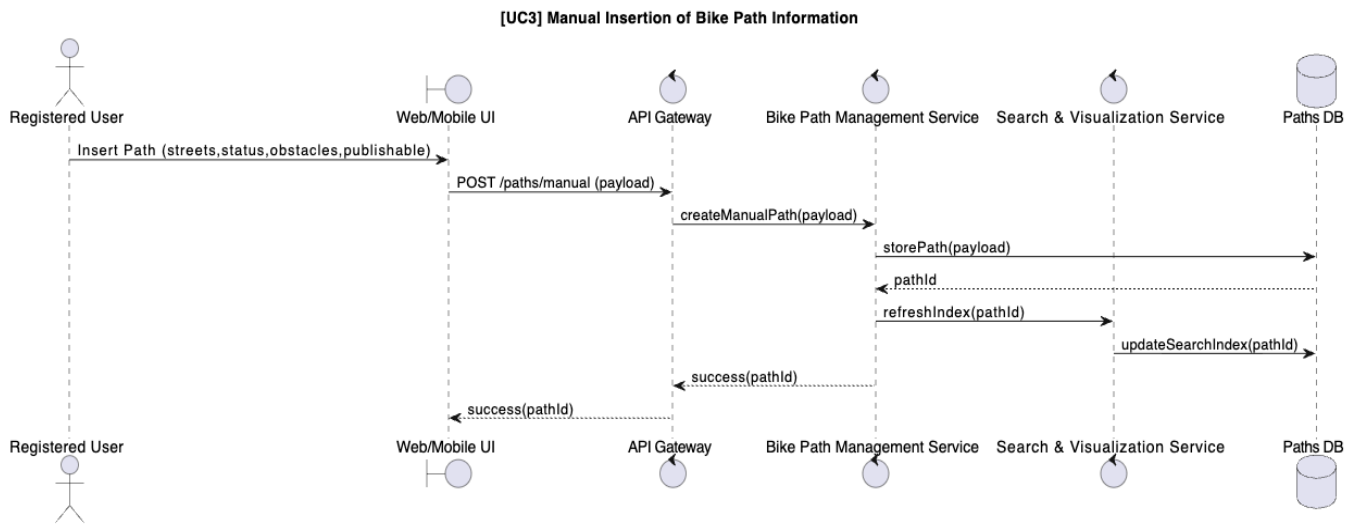
[UC1] Trip Recording (start/stop + GPS points + statistics)



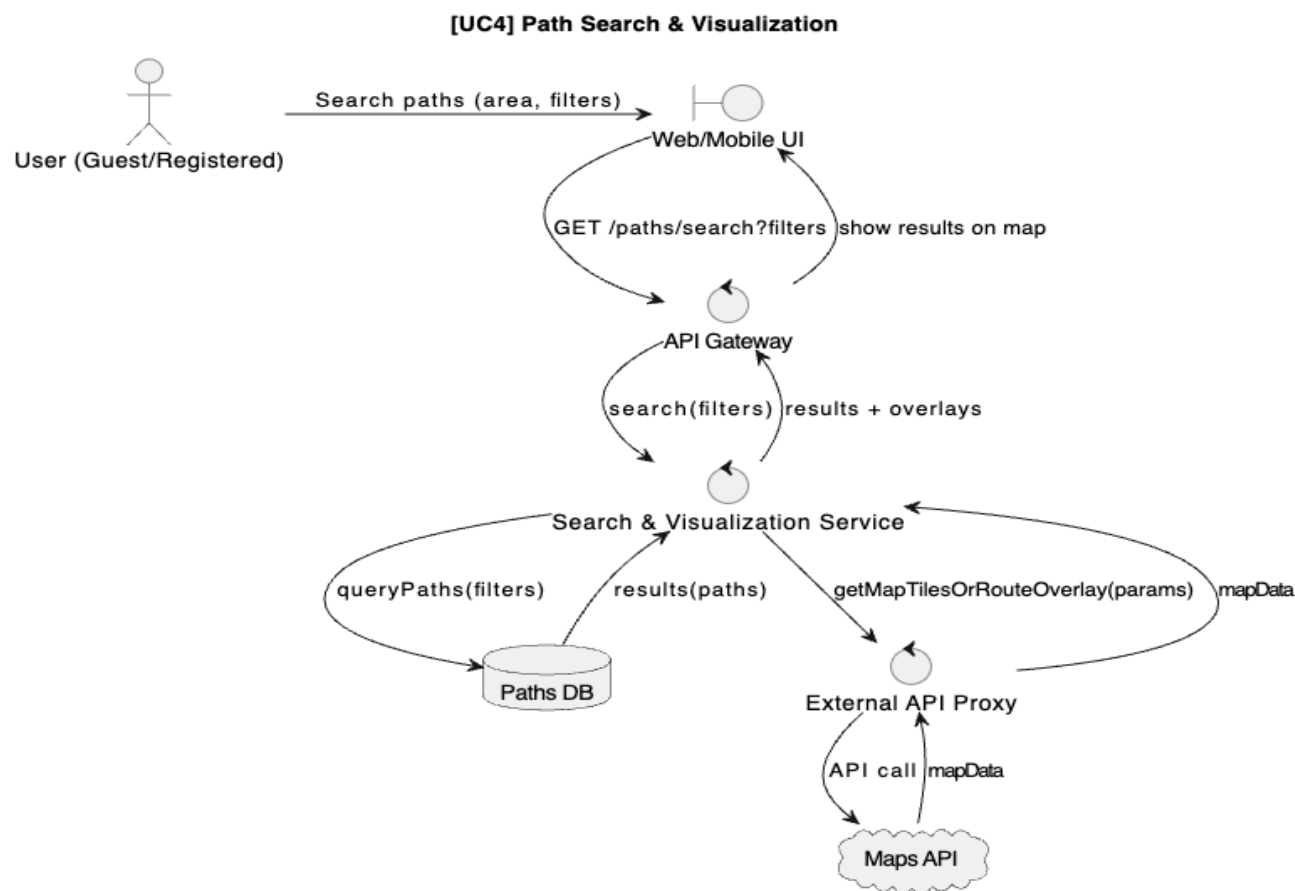
[UC2] Automated Bike Path Insertion (detect biking + confirmation)



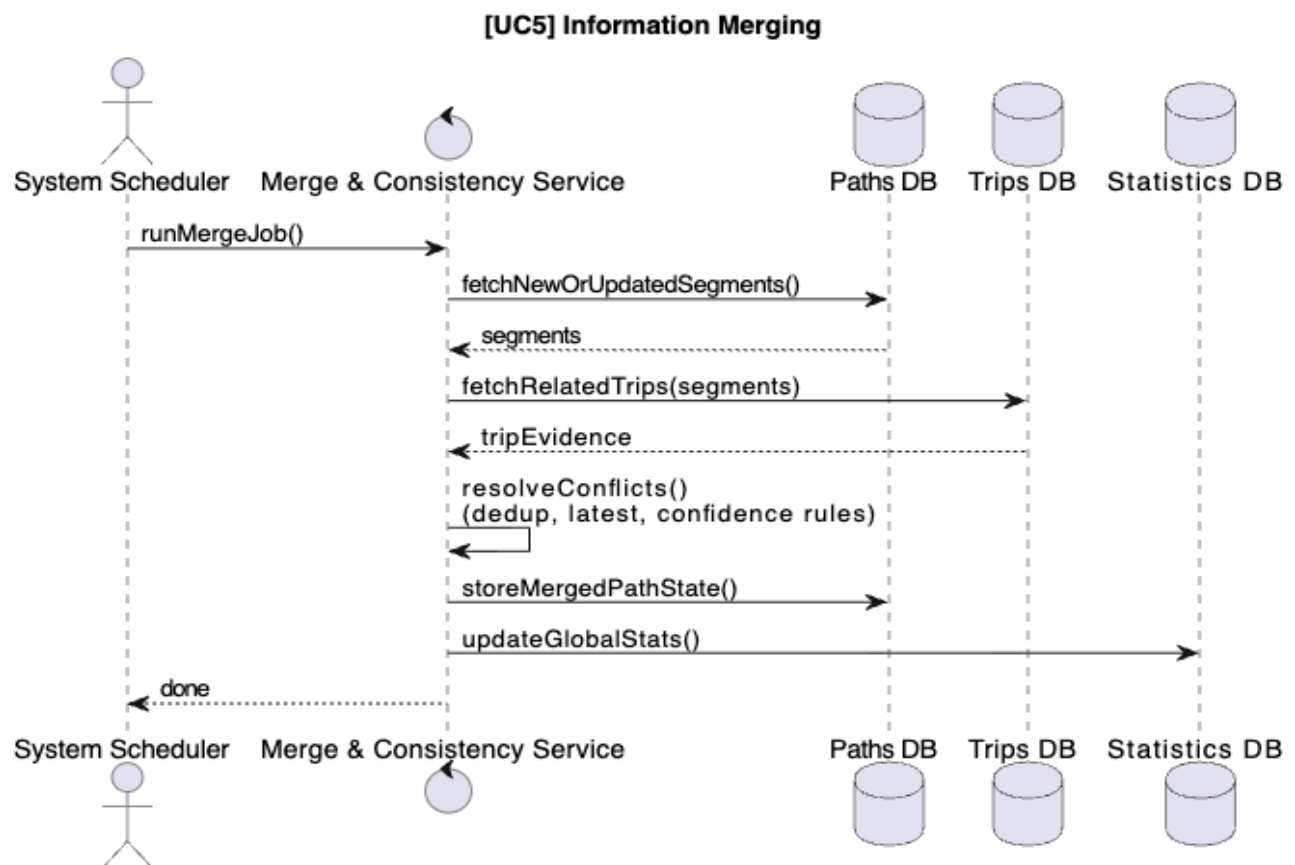
[UC3] Manual Insertion of Bike Path Information



[UC4] Path Search & Visualization



[UC5] Information Merging (conflicting/duplicate reports)



2.5 Component Interfaces

All internal BBP components communicate through **RESTful APIs** over HTTP(S). Each microservice exposes endpoints that follow standard HTTP conventions:

- **GET** operations retrieve data (e.g., searching paths, retrieving trip summaries);
- **POST** operations create new resources (e.g., new trip, manual path insertion);
- **PUT** operations update existing resources (e.g., profile updates, enabling automated mode).

For external services, BBP assumes provider APIs for:

- map visualization and overlays;
- meteorological data retrieval;
- push notification delivery.

All external calls are routed through the External API Proxy to centralize policy enforcement (e.g., rate limits, logging, retries).

2.6 Selected Architectural Styles and Patterns

- **Client–Server 4-Tier Architecture**
BBP uses a 4-tier separation to decouple user interaction from core services and from data storage, improving maintainability and security.
- **Microservices Architecture**
Core back-end functionalities are decomposed into independent services. This supports scalability and allows different parts of the system (e.g., trip ingestion vs. search) to scale independently.
- **REST Architectural Style**
Services expose REST endpoints to keep communication uniform and technology-agnostic.
- **Publish–Subscribe (for Notifications)**
Notification events (e.g., confirmation requests, system updates) can be handled using a publish–subscribe approach, where services publish events and the Notification Service delivers them to users through APNs/FCM.
- **Circuit Breaker Pattern (External APIs)**
To avoid cascading failures when external APIs are unavailable (maps/weather), circuit breakers can prevent repeated failing calls and allow the system to degrade gracefully.

2.7 Other Design Decisions

- **Proxy as Intermediary for External Services**
The External API Proxy acts as an intermediary for calls to maps, weather, and push providers. It provides centralized logging, caching of common responses (where applicable), and standardized handling of provider failures and rate limits.
- **Security**
All client-server communication uses HTTPS. Authentication is handled centrally via the API Gateway and Authentication Service. Data access is restricted so that each service can only operate on its relevant dataset. User-controlled publishability prevents unintended disclosure of private trip traces.
- **Availability and Fault Tolerance**
Replication of API Gateway and microservices supports high availability. Database replication and backup policies reduce the risk of data loss and allow failover in case of node failures.

3. User Interface Design

3.1 Overview

This chapter describes the **User Interface (UI) design** of the **Best Bike Paths (BBP)** mobile application. The UI has been designed to support cyclists in discovering, recording, and sharing bike paths while ensuring simplicity, usability, and safety during use. Since BBP is primarily a **mobile-based system**, the interface is optimized for smartphones and touch interaction. Special care has been taken to minimize user distraction during active trip recording, while still providing clear access to information before and after a trip. The UI design is consistent with the functional requirements described in the RASD and the architectural design presented in Chapter 2. Wireframes illustrate the main screens of the application and the navigation flow between them.

3.2 Design Principles

The BBP user interface follows the design principles listed below:

- **Mobile-first approach**
All screens are optimized for mobile devices and portrait orientation.
- **Minimal and neutral design**
The interface uses soft, neutral colors (light blue, light green, white, and gray) to ensure readability and reduce visual fatigue.
- **Map-centered interaction**
Maps play a central role in the application, as most features (trip recording, path visualization, insertion) are geographically driven.
- **Low interaction during movement**
During trip recording, user interaction is intentionally limited to essential actions (start and stop).
- **Consistency and clarity**
Similar UI components, icons, and layouts are reused across screens to reduce the learning curve.

3.3 Main Screens Description

This section describes the main screens of the BBP mobile application. Each subsection refers to a corresponding wireframe.

3.3.1 Welcome & Login Screen

The Welcome & Login screen is the entry point of the BBP application.

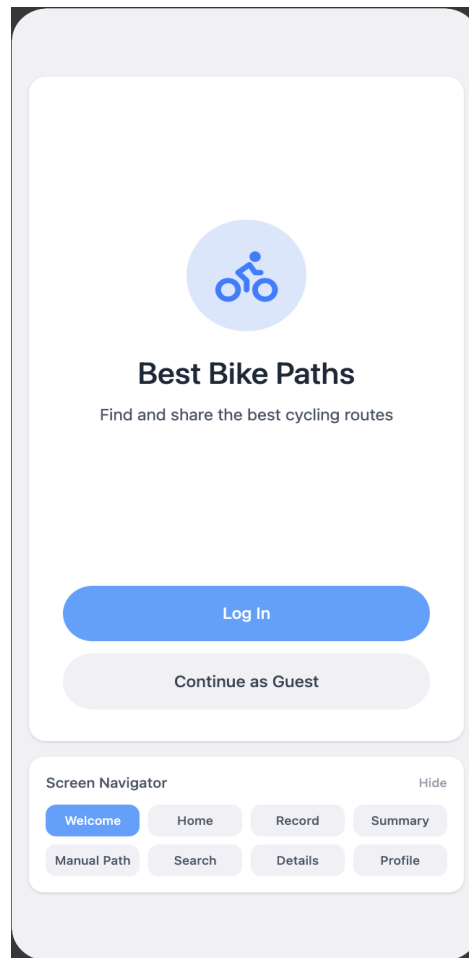
It allows users to:

- log in as **registered users**, or
- continue as **guest users**.

The screen presents:

- the application name and logo;
- a short description of the app's purpose;
- a primary **Log In** button;
- a secondary **Continue as Guest** button.

Guest users can explore published bike paths but cannot record trips or insert new path information.



3.3.2 Home / Map Screen

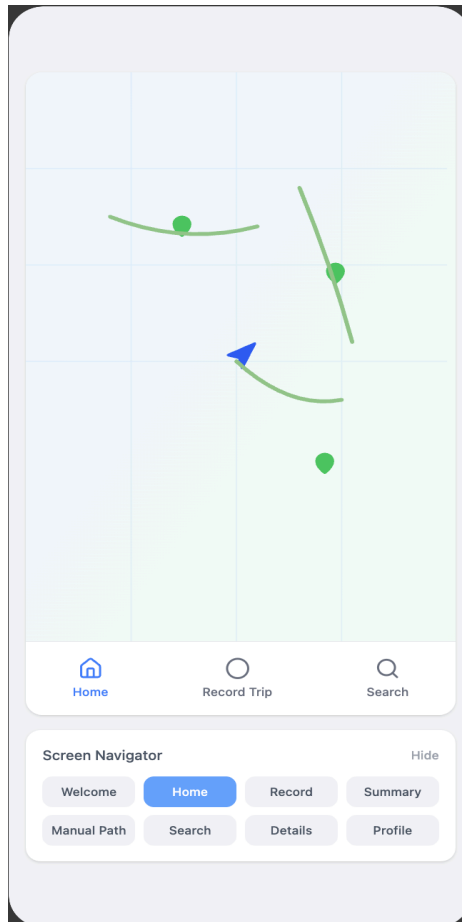
The Home screen represents the core navigation hub of the application. It displays:

- a full-screen interactive map;
- the user's current location;
- nearby bike paths highlighted on the map.

A bottom navigation bar provides quick access to:

- Home
- Record Trip
- Search

From this screen, users can easily start a new trip, explore paths, or move to other sections of the application.



3.3.3 Trip Recording Screen

The Trip Recording screen is shown when a registered user starts recording a bike trip.

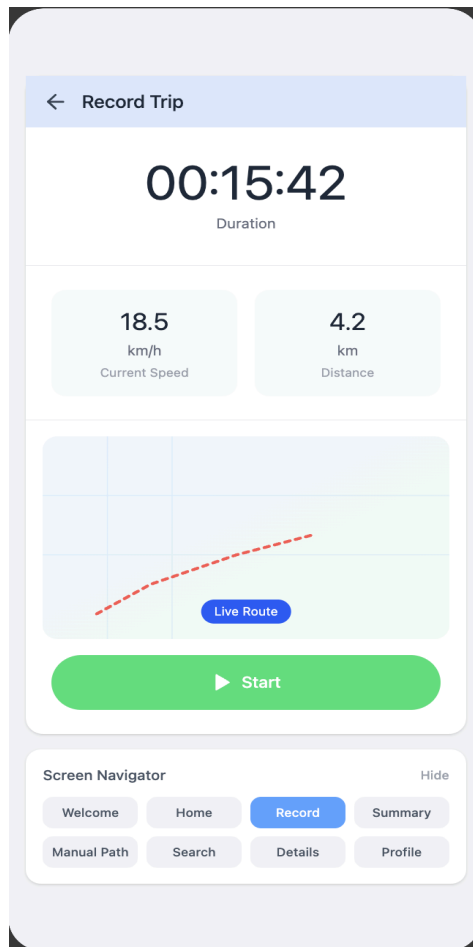
It displays:

- elapsed trip duration
- current speed
- total distance traveled
- a simplified map view showing the live route.

User interaction is limited to:

- **Start** (to begin recording)
- **Stop** (to end recording).

This design minimizes distraction and ensures safe usage while cycling.



3.3.4 Trip Summary Screen

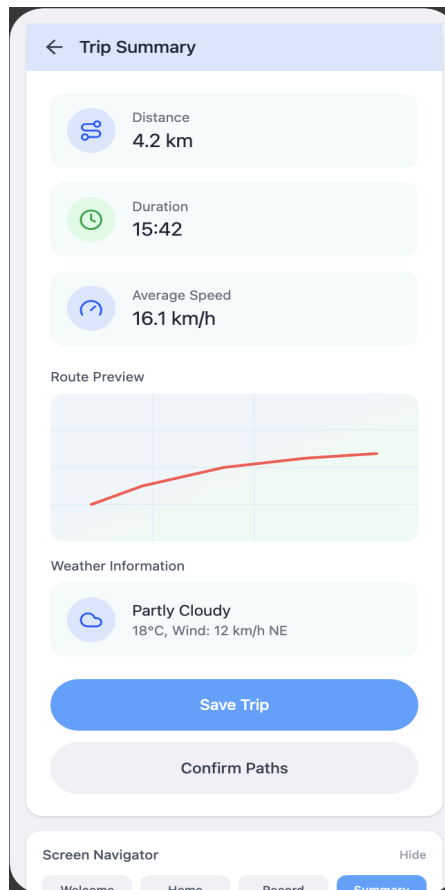
After a trip ends, the Trip Summary screen is presented.

It summarizes:

- total distance
- trip duration
- average speed
- a preview of the recorded route
- weather conditions associated with the trip.

From this screen, users can:

- save the trip
- confirm automatically detected bike path segments (when automated mode is enabled).



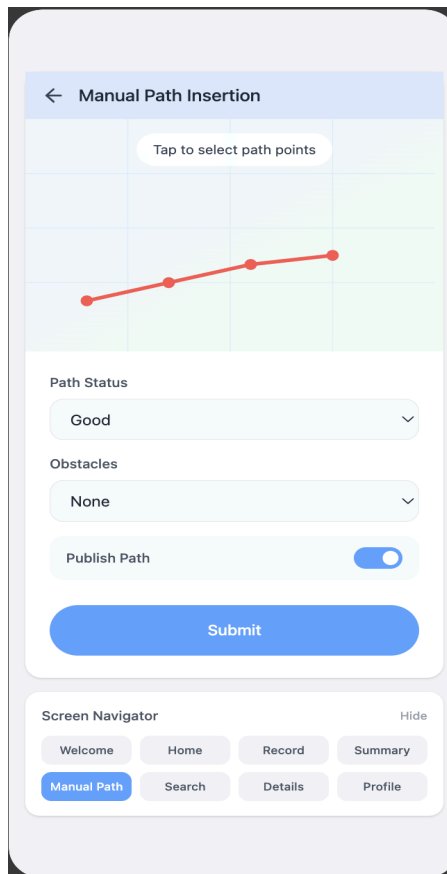
3.3.5 Manual Bike Path Insertion Screen

This screen allows registered users to manually insert bike path information.

It includes:

- a map where users can select path points.
- a dropdown menu to specify path status.
- a dropdown menu to report obstacles.
- a toggle to choose whether the path is publishable.

A **Submit** button sends the entered information to the system for storage and potential publication.



3.3.6 Search & Filter Screen

The Search & Filter screen enables users to explore bike paths based on specific criteria.

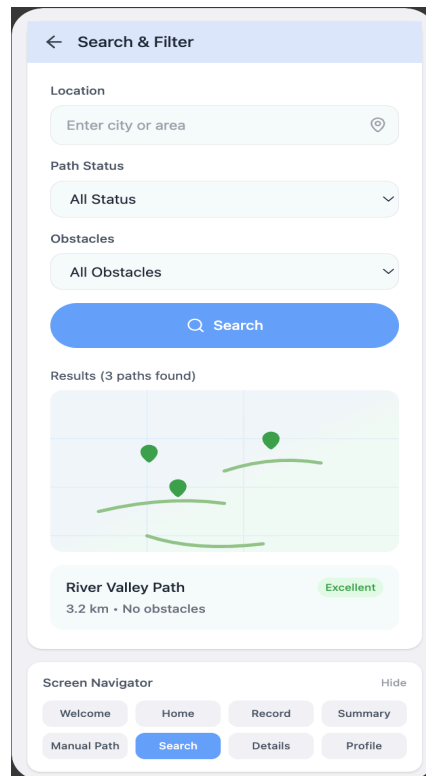
Users can:

- enter a city or area
- filter paths by status
- filter paths by obstacles.

Search results are shown both:

- visually on the map, and
- as summarized path cards below the map.

This screen is available to both guest and registered users.



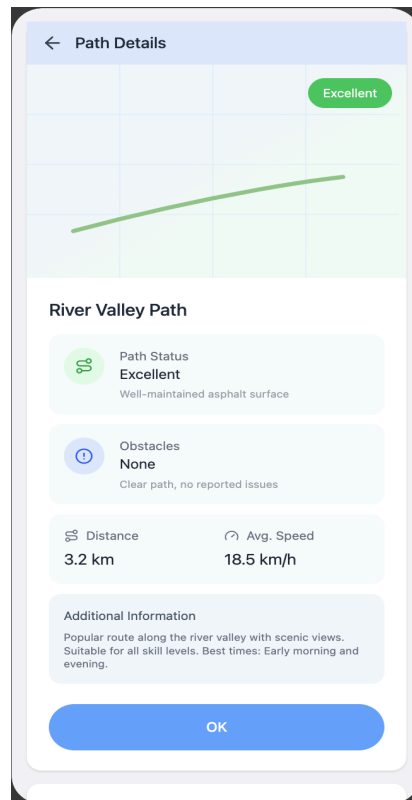
3.3.7 Bike Path Details Screen

The Bike Path Details screen provides detailed information about a selected bike path.

It displays:

- path name.
- path status.
- reported obstacles.
- total distance.
- average speed.
- additional descriptive information.

A map preview helps users visually understand the path. The screen is read-only for guest users.



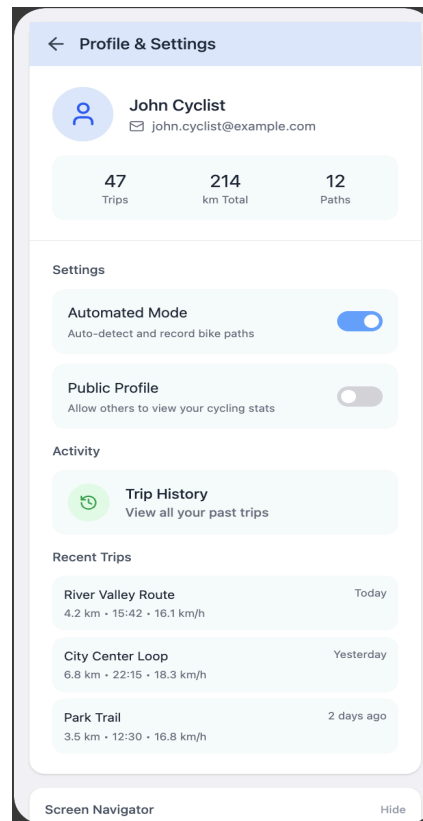
3.3.8 Profile & Settings Screen

The Profile & Settings screen is available only to registered users.

It allows users to:

- view profile information.
- check personal statistics (number of trips, total distance, contributed paths).
- enable or disable **automated mode**.
- control profile visibility.
- access trip history and recent activities.

This screen centralizes user-specific preferences and historical data.



3.4 Navigation Flow

Navigation within the BBP application is designed to be simple and intuitive.

- A **bottom navigation bar** provides access to the main sections (Home, Record Trip, Search).
- Back buttons are used for hierarchical navigation between screens.
- The user can always return to the Home/Map screen with minimal interaction.

This navigation structure ensures ease of use and consistency across the application.

3.5 UI Assumptions and Limitations

The proposed UI design is intended as a **conceptual representation** of the BBP mobile application.

The following assumptions apply:

- Visual styling and animations are kept minimal
- Accessibility features (e.g., font scaling, color contrast) are assumed to be handled during implementation
- Error handling and edge cases are not fully represented in the wireframes.

The focus of this chapter is on functional interaction, not on final visual aesthetics.

3.6 Wireframe Legend

In the provided wireframes:

- buttons represent user actions;
- toggles represent optional settings;

- maps represent interactive geographic elements;
- arrows and navigation components indicate screen transitions.

Each wireframe corresponds to one or more functional requirements described in the RASD.

4. Requirements Traceability

This section establishes a clear and verifiable mapping between the requirements defined in the Requirement Analysis and Specification Document (RASD) and the architectural and design elements presented in this Design Document (DD). The objective of requirements traceability is to ensure completeness, consistency, and accountability throughout the software engineering lifecycle, demonstrating that every critical requirement has been systematically addressed by one or more design decisions.

The traceability process follows a forward-traceability approach, linking each Functional Requirement (FR) and Non-Functional Requirement (NFR) from the RASD to the relevant subsystems, components, and design artifacts (e.g., modules, interfaces, and algorithms) described in this DD. This also enables impact analysis in case of future requirement changes.

4.1 Functional Requirements Traceability

No.	Description	Design Elements
1.	The system shall allow users to record cycling trips automatically.	Automated Mode, Quantified Self & Telemetry Module, Mobile Client Background Service.
2.	The system shall allow users to manually tag bike paths and hazards.	Crowdsourced Inventory Module, Mobile GUI, REST API (Path Annotation Endpoint).
3.	The system shall detect surface anomalies using smartphone sensors.	Smart Sensing Module, IMU Data Processing Pipeline, Anomaly Detection Logic.
4.	The system shall merge multiple reports referring to the same path segment.	Merging & Truth Maintenance Engine, Consensus Algorithm.
5.	The system shall visualize path quality on an interactive map.	Context-Aware Visualization Module, GIS Layer Manager, Path Score Renderer.
6.	The system shall compute safer routes based on path quality.	Routing Engine, Path Score Calculator, GIS Graph Weighting Mechanism.

4.2 Non-Functional Requirements Traceability

No.	Description	Design Elements
1.	Performance: The system shall process telemetry data in near real-time.	Asynchronous data ingestion, batch processing for sensor fusion
2.	Scalability: The system shall support a growing number of users.	Stateless REST services, horizontally scalable backend.
3.	Reliability: The system shall tolerate noisy or conflicting data.	Redundant data sources, consensus-based truth maintenance.
4.	Security & Privacy: The system shall protect user location data.	GDPR-compliant data handling, anonymization, access control.
5.	Usability: The mobile application shall be intuitive for cyclists.	Minimalist GUI, automated data collection, reduced user burden.

5. Implementation, Integration & Test Plan

5.1 Overview

This section outlines the planned strategy for implementing, integrating, and validating the BBP system. The approach follows an incremental and modular development methodology to reduce integration risks and ensure early validation of critical functionalities.

5.2 Implementation Plan

The implementation of the Best Bike Paths (BBP) system is structured around a set of subsystem-level milestones, each addressing a distinct functional responsibility within the overall architecture. This phased approach allows for incremental development and early validation of critical system components while minimizing integration risks.

The first implementation milestone focuses on the Backend Core Services, which constitute the backbone of the system. This phase includes the development of RESTful APIs responsible for user management, trip data uploads, and the handling of path metadata. In parallel, the geospatial database is designed and configured, including the definition of spatial schemas, indexing strategies, and data persistence mechanisms required to support efficient storage and querying of location-based information.

The second milestone addresses the Mobile Client Application, which serves as the primary interface for data acquisition and user interaction. This phase involves the implementation of GPS-based trajectory tracking and inertial data collection through the device's embedded IMU sensors. Additionally, both the Automated Mode background service and the manual

tagging interface are developed to enable opportunistic and participatory data collection, respectively, while ensuring minimal user interaction during normal cycling activities.

The Smart Sensing and Analytics milestone concentrates on transforming raw sensor data into meaningful infrastructure indicators. This includes the implementation of signal preprocessing techniques, such as filtering and normalization, to reduce sensor noise and variability. Building upon this preprocessing stage, anomaly detection heuristics are implemented and calibrated through threshold tuning to reliably identify surface irregularities, such as potholes, while minimizing false positives.

Finally, the Merging and Routing Engine milestone integrates the analytical results into actionable navigation intelligence. This phase involves the implementation of consensus algorithms to reconcile multiple, potentially conflicting user reports referring to the same path segment. Based on the merged data, Path Scores are computed and injected into the routing graph as weighted edges, enabling the system to generate routes that prioritize safety and ride quality over purely distance or time-based optimization.

5.3 Component Integration and Testing

Integration activities for the Best Bike Paths (BBP) system will follow a bottom-up strategy, emphasizing early validation of low-level components before progressing toward full system integration. Initially, individual components that have successfully passed unit testing such as sensor data collectors, signal preprocessing modules, database access layers, and routing utilities are integrated within their respective subsystems. This approach ensures that internal subsystem interactions are stable and that component-level dependencies are resolved in isolation before exposing functionality to higher architectural layers.

Once internal subsystem integration is achieved, subsystems are progressively integrated through well-defined RESTful interfaces. These interfaces act as contractual boundaries between the mobile client, backend services, and analytical engines, allowing each subsystem to evolve independently while preserving interoperability. Particular attention is given to the integration of data-intensive components, such as the interaction between telemetry ingestion services and the Merging & Truth Maintenance Engine, to ensure consistency in data formats, timing assumptions, and error-handling semantics.

During the early stages of integration, mock services and stubs are extensively employed to decouple frontend and backend development. For instance, simulated API endpoints and synthetic geospatial datasets are used to validate the mobile client's behavior in the absence of a fully deployed backend, while mock sensor data streams are utilized to test analytics components independently of real-world cycling conditions. This practice reduces integration bottlenecks and enables parallel development across teams.

As the system matures, incremental end-to-end integration is performed, gradually replacing mock components with their production counterparts. This staged integration process allows for controlled identification and resolution of integration defects, particularly those arising from asynchronous communication, network latency, or data synchronization issues.

To support this integration strategy, Continuous Integration (CI) pipelines are employed to automatically build, integrate, and validate the system upon each code commit. These pipelines execute automated unit and integration test suites, enforce coding and interface consistency, and provide immediate feedback to developers, thereby reducing regression risks and ensuring sustained architectural integrity throughout the development lifecycle.

5.4 Test Plan

Testing activities for the Best Bike Paths (BBP) system are organized according to a multi-layered testing strategy, designed to progressively validate the correctness, robustness, and safety of the system as integration advances. This structured approach ensures that defects are identified as early as possible while maintaining traceability between test cases and the requirements defined in the RASD.

Unit testing is performed at the lowest level of abstraction and focuses on the verification of individual software components in isolation. This includes testing core functions such as anomaly detection algorithms, Path Score computation logic, sensor data preprocessing routines, and database access operations. Unit tests are designed to validate both nominal behavior and boundary conditions, ensuring that each component behaves correctly under normal, extreme, and erroneous input scenarios. Mock objects and synthetic datasets are used extensively to isolate components and achieve deterministic test outcomes.

Integration testing validates the correct interaction between components and subsystems once unit-level correctness has been established. This testing phase focuses on the communication between the mobile client, backend services, and persistent storage systems. Particular attention is given to REST API interactions, data serialization and deserialization, and the consistency of geospatial data as it flows through the system. Integration tests are also used to identify issues related to asynchronous processing, network latency, and partial system failures, ensuring that subsystem boundaries are robust and well-defined.

System testing evaluates the BBP system as a complete, fully integrated product in an environment that closely resembles real-world deployment conditions. End-to-end test scenarios are defined to reflect realistic user workflows, such as automated trip logging, background anomaly detection, manual hazard confirmation, and safety-aware route computation. These tests verify that functional and non-functional requirements—including performance, reliability, and usability—are collectively satisfied when the system operates as a whole.

Finally, User Acceptance Testing (UAT) is conducted with representative cyclists to validate that the system meets user expectations and delivers tangible safety and usability benefits. UAT focuses on qualitative aspects such as ease of use, clarity of map visualizations, perceived accuracy of hazard reporting, and overall confidence in route recommendations.

6. Effort Spent

The following table displays the number of hours each group member spent on the various sections of the document. Please note that the division is only approximate, and each section still required the collaboration of all members.

Group Member	Detail of Work	Hours Spent
Nafis Ismam	Introduction and User Interface Design: Definition of scope, design goals, and creation of UI. Requirements Traceability: Mapping of RASD requirements to design components. Implementation, Integration & Testing: Definition of the development strategy, testing phases, and integration plan. Final Documentation: Compilation of the final document, including a list of references and an effort report.	16
Ayman Mansur	Architectural Design: High-level component definition and architectural views. User Interface Design: Design and final version of the UI.	16
Christopher Khoury	Overall Description: Analysis and definition of the system context, user characteristics, and domain assumptions. User Interface Design: Design of the user interface and creation of the UI.	14

7. References

- [1] N. Ismam, A. Mansur, C. Khoury, “*Requirement Analysis and Specification Document (RASD) – Best Bike Paths*,” Version 2.0, Dec. 2025.
- [2] ISO/IEC/IEEE, “*Systems and Software Engineering — Design Description*,” ISO/IEC/IEEE Standard 1016:2009.
- [3] IEEE, “*IEEE Recommended Practice for Software Requirements Specifications*,” IEEE Std 830-1998.
- [4] M. F. Goodchild, “*Citizens as Sensors: The World of Volunteered Geography*,” *GeoJournal*, vol. 69, no. 4, pp. 211–221, 2007.
- [5] OpenStreetMap Foundation, “*OpenStreetMap Wiki – Map Matching and GIS Concepts*,” available online.
- [6] Google Developers, “*Android Sensor Framework Documentation*,” available online.
- [7] European Union, “*General Data Protection Regulation (GDPR)*,” Regulation (EU) 2016/679, Official Journal of the European Union.